

MA653P - Computational Financial Modelling Lab

Assignment 5

Instructions.

- This assignment builds upon the dataset collected in Assignment 1 and processed in Assignment 2. The Portfolio models used in Assignment 4 must be solved using the heuristic optimization. Students must **ONLY** use the data of N stocks gathered earlier for their selected index.
- Use initial 70% part of the data as a seen data for model building (finding VAR, CVAR, portfolio construction etc.) and remaining 30% data for testing the performance.
- Submit a single `.ipynb` Jupyter notebook named `RollNo_MA653P_A05.ipynb`.
- Every cell must be executed top-to-bottom with no errors before submission. Include all output (tables, plots, printed values) in the saved notebook.
- Use **only** the following libraries: `numpy`, `scipy`, `pandas`, `matplotlib`, `seaborn`, `yfinance`, `cvxpy`. All are installable via `pip`.
- Each task must begin with a **Markdown cell** stating the task number, title, and a brief description of your approach.
- Figures must have titles, axis labels, and legends. Tables must have column headers.
- The evaluation will be based on **correctness**, **clarity of code**, **quality of output**, **written observations**, and **responses to the questions asked**.
- The goal is not just code that runs, but code that *explains* — through outputs, plots, and commentary.

-
1. Solve the **same three portfolio models** (Minimum Variance, CVaR, Semivariance) using a **Genetic Algorithm (GA)** implemented with any available online tool or library of your choice (e.g. DEAP, PyGAD, `geneticalgorithm`, or a custom NumPy implementation).

GA parameters to use (justify any deviation in a Markdown cell):

- Population size: $P = 200$ solutions
- Generations: $G = 500$
- solution: real-valued vector $\mathbf{w} \in \mathbb{R}^N$ (one gene per asset weight)
- Crossover: Simulated Binary Crossover (SBX), $\eta_c = 15$, crossover probability $p_c = 0.9$
- Mutation: Polynomial mutation, $\eta_m = 20$, mutation probability $p_m = 1/N$
- Selection: Binary tournament selection
- Elitism: carry the best solution of the current population by replacing with the worst into the next generation
- Fitness: each objective (variance, CVaR, semivariance) evaluated *after* applying the full repair sequence

2. Modify the above model by adding the **budget**, **bound**, and **cardinality** constraints. To solve the modified constrained model update the GA to simultaneously satisfy the constraints using the following repair mechanism.

(a) Repair mechanism — Budget constraint.

After crossover and mutation the solution $\mathbf{w}$ may have negative entries or fail to sum to one. Implement the following budget repair:

$$\tilde{w}_i = \frac{\max(w_i, 0)}{\sum_{j=1}^N \max(w_j, 0)}, \quad i = 1, \dots, N.$$

This clips each weight to zero (enforcing long-only) and re-normalises so the portfolio sums to one.

Edge case: if all genes are ≤ 0 , assign equal weights $\tilde{w}_i = 1/N$.

Write the function `repair_budget(w)` and verify it on three hand-crafted test vectors (one with negative entries, one that does not sum to 1, one already feasible).

(b) **Repair mechanism — Bound constraints.**

Each weight must satisfy $\ell_i \leq w_i \leq u_i$. Use the following two-stage bound repair applied *after* the budget repair:

$$w_i \leftarrow \min(\max(w_i, \ell_i), u_i), \quad \tilde{w}_i = \frac{w_i}{\sum_{j=1}^N w_j}.$$

Apply this to the constraint set: $\ell_i = 0.02$ (minimum 2% in any held asset) and $u_i = 0.30$ (maximum 30% in any single asset).

- i. Write the function `repair_bounds(w, l, u)`.
- ii. Show that after repair $\ell_i \leq \tilde{w}_i \leq u_i$ for all i and $\sum_i \tilde{w}_i = 1$. Verify with a printed example.
- iii. Explain in a Markdown cell why bound repair must be applied *before* cardinality repair but *after* the initial budget repair. What goes wrong if the order is reversed?

(c) **Repair mechanism — Cardinality constraint.**

At most $K = \lceil N/3 \rceil$ assets may be held (non-zero weight). Apply the following cardinality repair *last* in the repair chain:

$$\begin{aligned} \text{rank weights descending: } w_{(1)} \geq w_{(2)} \geq \dots \geq w_{(N)}, \\ \tilde{w}_i = \begin{cases} w_i & \text{if } w_i \text{ is among the top-}K \text{ weights,} \\ 0 & \text{otherwise,} \end{cases} \quad \hat{w}_i = \frac{\tilde{w}_i}{\sum_j \tilde{w}_j}. \end{aligned}$$

- i. Write the function `repair_cardinality(w, K)`.
- ii. Write the **combined repair pipeline**:

$$\begin{aligned} \text{repair_all}(w, K, l, u) \equiv \\ \text{repair_cardinality}(\text{repair_bounds}(\text{repair_budget}(w), l, u), K) \end{aligned}$$

Call `repair_all` on a random 20-element vector and print the output, confirming: (i) sum = 1.0, (ii) all non-zero weights are in $[\ell, u]$, and (iii) at most K weights are non-zero.

(d) **GA fitness function.**

Write a unified fitness function `fitness(w, model, R, alpha, tau)` that:

- i. Calls `repair_all(w, K, l, u)` to obtain a feasible solution.
- ii. Evaluates the objective:

$$f(w) = \begin{cases} w^\top \hat{\Sigma} w & \text{M1 — Minimum Variance,} \\ \hat{\zeta} + \frac{1}{(1-\alpha)T} \sum_{t=1}^T u_t & \text{M3 — CVaR (scenario estimate),} \\ \frac{1}{T} \sum_{t=1}^T d_t^2 & \text{M4 — Semivariance.} \end{cases}$$

- iii. Returns $f(w)$ as the value to be **minimised** (lower is better in all three cases).

Note: for M3, compute $\hat{\zeta} = \widehat{\text{VaR}}_\alpha(w)$ from the scenario losses $-Rw$, then

(e) **Run the GA for all three models.**

For each model :

- i. Run the GA using the training data (initial 70%).
- ii. Plot the **convergence curve**: best fitness value vs. generation number. Label the final fitness, the generation at which the best solution was first found, and mark any plateaux.

- iii. Print the final optimal weight vector (non-zero weights only), the selected stock names, and the in-sample performance metrics: μ_P , σ_P , SR, $\text{VaR}_{99\%}$, $\text{CVaR}_{99\%}$.
3. Compare the GA solutions for unconstrained models in Task 1 against the and constrained portfolio in Task 2 above. All comparisons are made on the **out-of-sample test set**.

(a) **Out-of-sample performance table.**

For each of the six portfolio variants below, compute the out-of-sample performance metrics defined and fill in the table.

Variant	μ_P	σ_P	SR	$\text{CVaR}_{99\%}$
Model 1 (unconstrained)				
Model 1 (budget + bounds + cardinality)				
Model 2 (unconstrained)				
Model 2 (budget + bounds + cardinality)				
Model 3 (unconstrained)				
Model 3 (budget + bounds + cardinality)				

Submission checklist.

- ☐ Notebook runs end-to-end without errors
- ☐ `repair_budget`, `repair_bounds`, `repair_cardinality`, `repair_all` functions implemented and tested
- ☐ GA convergence curves plotted for all three models
- ☐ File named `RollNo_MA653P_A05.ipynb`